

Restaurant Order Board

Project Synopsis

Team Details : TEAM 10 – PSV (Public Static Void)

Sameeksha KC | Sanat Shirwaikar | Shreyansh Shubham | Siddharth M

PES University – UG23AM

1. Executive Summary

This document outlines the design and implementation strategy for the **Restaurant Order Board** module (Team 10). The primary goal is to provide kitchen staff with a real-time, filtered view of active orders and to manage the kitchen-side status lifecycle — from order confirmation through preparation to ready-for-pickup — while enforcing strict transition rules and rejecting illegal status changes.

The module integrates with the Order Orchestration module (Team 9) via the *OrderRepository* interface and is fully testable through an in-memory repository implementation.

2. Requirements to Satisfy

To ensure proper functionality and adherence to system architecture, the following requirements must be met:

- **Active Order Visibility:** The board must display only orders whose status is `CONFIRMED`, `PREPARING`, or `READY_FOR_PICKUP`.
- **Status Transitions:** Kitchen staff can advance orders along the path: `CONFIRMED` → `PREPARING` → `READY_FOR_PICKUP`.
- **Illegal Transition Rejection:** Any transition outside the allowed path must throw an *IllegalStateException* with a descriptive message.
- **Restaurant Isolation:** Each restaurant sees only its own orders — cross-restaurant data must never be exposed.
- **Deterministic Ordering:** Active orders are returned sorted by *orderId* for a stable, predictable board display.
- **Integration:** Must depend only on *OrderRepository* from the *edu.classproject.order* package.
- **Compliance:** Must pass all unit tests and expose a clear service contract via the *RestaurantOrderBoardService* interface.

3. Proposed Implementation

The implementation follows a clean, dependency-inverted architecture within the package `edu.classproject.restaurantops`.

3.1 Components

- **RestaurantOrderBoardService (Interface):** Defines the public contract — fetching active orders and performing status transitions.
- **DefaultRestaurantOrderBoardService (Implementation):** Contains core business logic — filtering by restaurant and status, stable sorting, and enforcing state-machine rules.
- **Dependencies:** *OrderRepository* (Team 9) — to look up, update, and persist order records.

3.2 Service Contract (Interface Methods)

Method	Signature	Description
<code>getActiveOrders</code>	<code>getActiveOrders(String restaurantId) → List<Order></code>	Returns all <code>CONFIRMED</code> / <code>PREPARING</code> / <code>READY_FOR_PICKUP</code> orders for the given restaurant, sorted by <code>orderId</code> .
<code>markPreparing</code>	<code>markPreparing(String orderId) → void</code>	Transitions order from <code>CONFIRMED</code> to <code>PREPARING</code> . Throws <code>IllegalStateException</code> for any other current status.
<code>markReadyForPickup</code>	<code>markReadyForPickup(String orderId) → void</code>	Transitions order from <code>PREPARING</code> to <code>READY_FOR_PICKUP</code> . Throws <code>IllegalStateException</code> for any other current status.

3.3 Active Status Set

The following *OrderStatus* values are considered 'active' — meaning the kitchen still owns these orders and they appear on the board:

Status	Meaning
<code>CONFIRMED</code>	Order accepted by restaurant; queued for preparation.
<code>PREPARING</code>	Kitchen is actively preparing the order.
<code>READY_FOR_PICKUP</code>	Order is complete; awaiting dispatcher assignment.

4. Functional Behavior

Feature	Description
<code>getActiveOrders</code>	Filters all orders by <code>restaurantId</code> and the active status set; returns a stable list sorted by <code>orderId</code> .

Feature	Description
<code>markPreparing</code>	Validates current status is CONFIRMED, then sets it to PREPARING and persists via <code>orderRepository.save()</code> .
<code>markReadyForPickup</code>	Validates current status is PREPARING, then sets it to READY_FOR_PICKUP and persists via <code>orderRepository.save()</code> .
Illegal transition	Any status not matching the expected value throws <code>IllegalStateException</code> with a message containing both the current and expected statuses.
Order not found	If <code>findById()</code> returns an empty <code>Optional</code> , throws <code>IllegalArgumentException("Order not found: ")</code> .

5. Validation Rules

Field / Input	Validation Requirement
<code>restaurantId</code>	Must be non-null and non-blank; blank value throws <code>IllegalArgumentException</code> .
<code>orderId</code>	Must correspond to an existing order; not-found throws <code>IllegalArgumentException</code> .
Status for <code>markPreparing</code>	Must be CONFIRMED; any other status throws <code>IllegalStateException</code> .
Status for <code>markReadyForPickup</code>	Must be PREPARING; any other status throws <code>IllegalStateException</code> .
<code>OrderRepository</code> (constructor)	Null value throws <code>IllegalArgumentException</code> — fail-fast at construction time.
Consistency Rule	Given the same repository state, <code>getActiveOrders()</code> always returns the same sorted list.

6. Test and Demo Plan

6.1 Unit Tests

- **Happy Path – `getActiveOrders`:** Verify that CONFIRMED, PREPARING, and READY_FOR_PICKUP orders are all returned for the correct restaurant.
- **Happy Path – `markPreparing`:** A CONFIRMED order transitions to PREPARING and the change is persisted in the repository.
- **Happy Path – `markReadyForPickup`:** A PREPARING order transitions to READY_FOR_PICKUP and the change is persisted.
- **Edge 1 – `markPreparing` wrong status:** Calling `markPreparing` on a PREPARING order throws `IllegalStateException` whose message contains both 'PREPARING' and 'CONFIRMED'.
- **Edge 2 – `markReadyForPickup` wrong status:** Calling `markReadyForPickup` on a CONFIRMED order throws `IllegalStateException`.

- **Edge 3 – Unknown orderId:** `markPreparing` on a non-existent `orderId` throws `IllegalArgumentException`.
- **Edge 4 – Unknown restaurant:** `getActiveOrders` for an unregistered `restaurantId` returns an empty, non-null list.
- **Edge 5 – Cross-restaurant isolation:** Orders from Restaurant B do not appear in Restaurant A's active list.
- **Edge 6 – Non-active status exclusion:** `DELIVERED`, `CANCELLED`, and `PAYMENT_FAILED` orders are excluded from the board.
- **Full kitchen flow:** End-to-end test: `CONFIRMED` → `PREPARING` → `READY_FOR_PICKUP` → removed from board after dispatcher picks up (order moves to `OUT_FOR_DELIVERY`).

6.2 Demo Scenario

1. Create three orders for Restaurant A: one `CONFIRMED`, one `PREPARING`, one `READY_FOR_PICKUP`.
2. Call `getActiveOrders('rest-A')` — verify all three appear, sorted by `orderId`.
3. Call `markPreparing('ord-1')` — verify status changes to `PREPARING` in the repository.
4. Call `markReadyForPickup('ord-1')` — verify status changes to `READY_FOR_PICKUP`.
5. Simulate dispatcher moving the order to `OUT_FOR_DELIVERY` (external team action on the repository).
6. Call `getActiveOrders('rest-A')` again — verify the dispatched order no longer appears.
7. Attempt `markPreparing` on a `READY_FOR_PICKUP` order — verify `IllegalStateException` is thrown.

7. Proposed Class Diagram

The module follows a dependency-inversion pattern. **DefaultRestaurantOrderBoardService** implements **RestaurantOrderBoardService** and depends only on the **OrderRepository** interface (Team 9). No concrete classes from other modules are referenced directly.

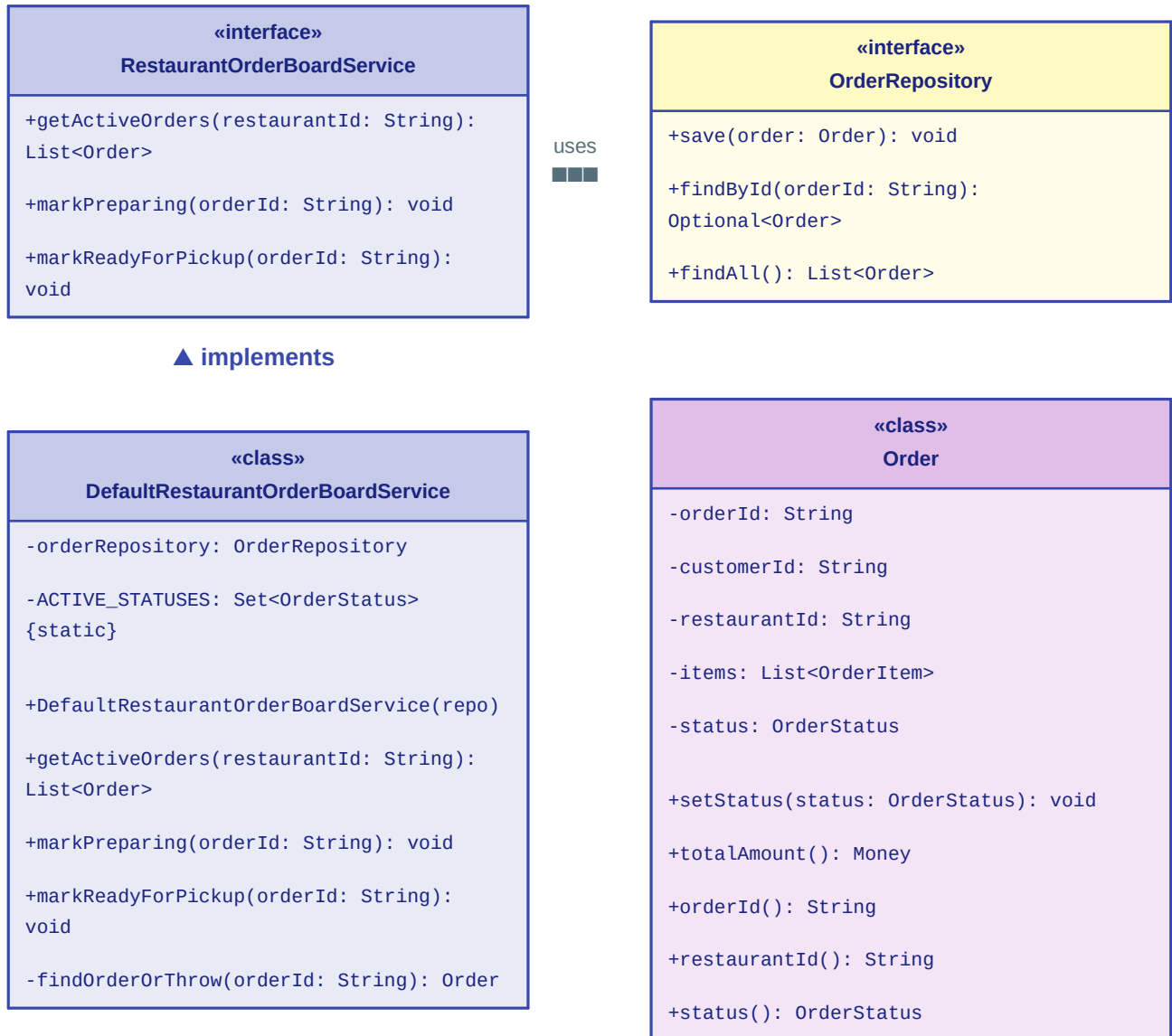


Diagram Legend

Notation	Meaning
▲ implements	Implements interface (realization)
uses ■■■■■■	Depends on / uses
«interface»	Java interface
«class»	Concrete implementation class

8. Sequence Diagram – Kitchen Status Update Flow

The following table describes the full message flow when a kitchen operator calls `markPreparing(orderId)` on a confirmed order:

#	Actor	Target	Message / Action
1	KitchenUI	DefaultRestaurantOrderBoardService	<code>markPreparing(orderId)</code>
2	DefaultRestaurantOrderBoardService	OrderRepository	<code>findById(orderId)</code>
3	OrderRepository	DefaultRestaurantOrderBoardService	Returns <code>Optional<Order></code>
4	DefaultRestaurantOrderBoardService	(self – validation)	Check: <code>order.status() == CONFIRMED</code> → if not, throw <code>IllegalStateException</code>
5	DefaultRestaurantOrderBoardService	Order	<code>order.setStatus(PREPARING)</code>
6	DefaultRestaurantOrderBoardService	OrderRepository	<code>orderRepository.save(order)</code>
7	DefaultRestaurantOrderBoardService	KitchenUI	return (void)

9. Integration Points & Assumptions

- **Team 9 – Order Orchestration:** Provides `OrderRepository`, `Order`, and `OrderItem`. This module never constructs orders; it only reads and updates their status.
- **Team 1 – User Identity:** Customer and restaurant IDs are assumed to be pre-validated by upstream modules before reaching this service.
- **Team 13 – Live Tracking / Dispatch:** Once an order moves to `READY_FOR_PICKUP`, the dispatcher is expected to advance it to `OUT_FOR_DELIVERY`, removing it from the kitchen board.
- **OrderStatus enum:** Assumed to contain at minimum: `CONFIRMED`, `PREPARING`, `READY_FOR_PICKUP`, `OUT_FOR_DELIVERY`, `DELIVERED`, `CANCELLED`, `PAYMENT_FAILED`.
- **Thread safety:** The current implementation is not thread-safe. Concurrent updates to the same order should be handled at the repository layer in production.
- **No timestamps on Order:** Stable ordering is achieved via natural `orderId` string comparison as a proxy for insertion order, as the `Order` record exposes no creation timestamp.